Vehículo esquivador de obstáculos

Sistemas Digitales(IF471A)

Gredmar Miguel Ulloa Gómez, gredmar.ulloa.g@uni.pe
Gustavo Lescano Salas, glescanos@uni.pe

Universidad Nacional de Ingeniería, Facultad de Ciencias
18 de diciembre de 2025

Resumen

Para el desarrollo del proyecto final mostrado en el presente informe se usó una placa de Arduino UNO R3 para controlar un sensor ultrasonido HC-SR04, un servomotor SG90, dos motores-reductores de doble eje y de dos puentes H (L298N); además de un chasis, un par de ruedas y baterías de 9V. Aplicando dichos conceptos y materiales se desarrolló un mini-vehículo esquivador de obstáculos, que se compone de 3 etapas, *adquisión de datos* que consiste la distancia que mide a sus alrededores; *procesamiento de datos*, mediante un código de C++ desarrollado en el IDE de arduino se condicionan los posibles movimientos del vehículo; y *control*, donde controla la dirección del vehículo para evitar colisinar con su entorno.

1. Objetivos

- Aplicar los conceptos aprendidos y equipos usados en el curso, buscando la manera de crear un mini-vehículo esquivador de obstáculos.
- Desarrollar las etapas de adquisición de datos, procesamiento y control en un vehículo esquivador de obstáculos.

2. Fundamento Teórico

2.1. Arduino UNO R3:

El Arduino es una tarjeta de desarrollo basada en el microcontrolador Atmega 328p y viene preprogramado con un gestor de arranque que le permite cargar un nuevo código via USB sin el uso de un programador de hardware externo. La placa de Arduino mostrada en la figura 1 trabaja con una aplicación multiplataforma que esta escrita en lenguaje de programación Java. Se utiliza para escribir y cargar programas en placas compatibles con Arduino , admite los lenguajes C y C++.

Tambien esta aplicación incorpora un conjunto de librerías que proporcionan funciones para el control de los puertos de entrada y salida del microcontrolador. Entre sus características principales se encuentran las siguientes:

- **Microcontrolador:** ATmega328P
- **Voltaje de operación:** 5V
- **Voltaje de entrada:** [7-12]V
- **Pines digitales de entrada/salida:** 14
- **Entradas analógicas:** 6
- **Corriente máxima por pin:** 40mA
- **Frecuencia de reloj de cerámica:** 16MHz
- **Interfaces de comunicación:** UART, SPI, I2C
- **Conectividad USB:** Para la programación y alimentación de la placa

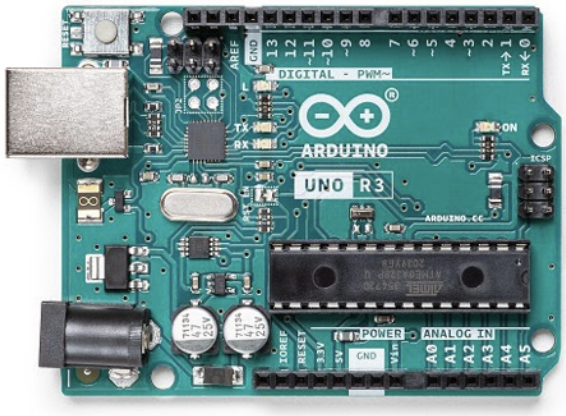


Figura 1: Placa de Arduino UNO R3.[1]

2.2. Señales PWM:

Como se indicó anteriormente, el Arduino UNO dispone de 6 salidas PWM de 8 bits, esta señal consiste en una técnica utilizada para crear una señal digital que puede simular un valor analógico ($V_{promedio}$) mediante el control de la proporción del tiempo durante el cual la señal está en nivel alto (T_{pulse}) frente al tiempo en el cual está en nivel bajo durante un ciclo (T), tal como se puede observar en la figura 2.

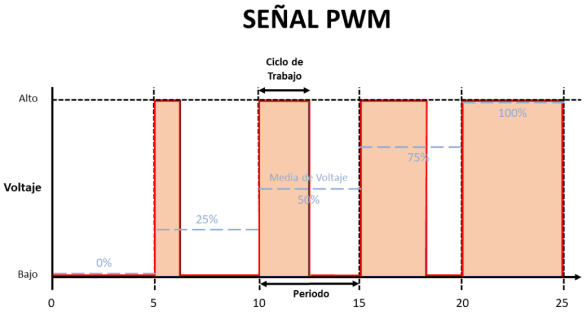


Figura 2: Señales PWM con diferentes periodos de pulso (trabajo). [2]

$$\text{Ciclo de trabajo} = \left(\frac{T_{\text{pulso}}}{T}\right) \times 100$$

T: periodo

$$V_{\text{promedio}} = 5\left(\frac{\text{Ciclo de trabajo}}{100}\right)$$

2.3. Servomotor SG90:

Un servomotor es un actuador rotativo que permite un control preciso de la posición angular, la velocidad y la aceleración, consiste en un motor adecuado acoplado a un sensor para la retroalimentación de la posición; a menudo está equipado con un controlador sofisticado, generalmente un módulo dedicado diseñado específicamente para su uso con servomotores.

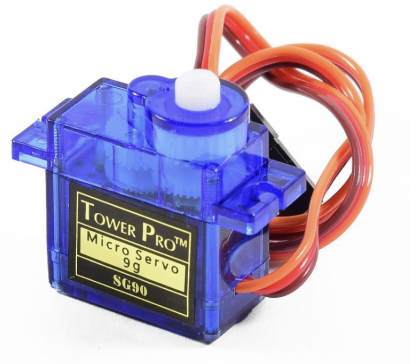


Figura 3: Imagen referencial de un micro servomotor SG90. [3]

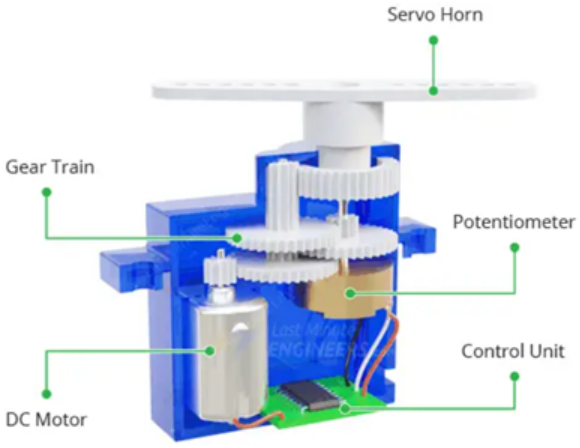


Figura 4: Partes internas del servomotor SG90. [4]

El servomotor mostrado en la figura 3 es de un tamaño compacto, pequeño y ligero; y funciona con corriente DC, por lo que también puede ser controlado por señales PWM (Modulación por ancho de pulso), dependiendo del tiempo que la señal del

pulso esté en nivel alto ($[0.5 - 2.4]ms$) en su periodo ($20ms$) se obtiene mayor valor en la corriente promedio total y el servomotor girará desde 0 a 180°.

Especificaciones:

- **Ángulo de Rotación:** Aproximadamente 180 grados (90 grados en cada dirección desde el punto central).
- **Voltaje de Funcionamiento:** [4.8 - 6.0]V
- **Par Motor:** 1.8 kg/cm (4.8V) - 2.2 kg/cm (6.0V)
- **Velocidad:** 4.8V: 0.12 segundos/60 grados - 6.0V: 0.10 segundos/60 grados
- **Dimensiones:** 22.2mm x 11.8mm x 31mm.
- **Conectores:** 3 pines de conexión(control, potencia y tierra).

2.4. Sensor ultrasónico HC-SR04:

El sensor ultrasónico HC-SR04 es un dispositivo electrónico que sirve para medir distancias. Su transductor se basa en el efecto piezoelectrico, el cual consiste en la generación de corriente en un material pierzoeléctrico mediante vibraciones de resonancia específicas (resonancia natural del material), estas vibraciones serán generadas por el frente de onda de llegada que interactúa con el material, haciéndolo vibrar.



Figura 5: Pines de conexión para el sensor ultrasonido HC-SR04. [5]

El sensor funciona con la emision pulsos ultrasónicos (rango de 40kHz) y midiendo el tiempo que tardan en regresar después de que su frente de onda se refleja en un objeto.

Especificaciones

- **Voltaje de Operación**: 5V DC
- **Corriente de Operación**: 15 mA
- **Frecuencia Ultrasónica**: 40 kHz
- **Rango de Detección**: [2 - 400]cm
- **Precisión**: Aproximadamente 3mm
- **Ángulo de apertura**: 15°
- **Dimensiones**: 45 mm x 20 mm x 15 mm

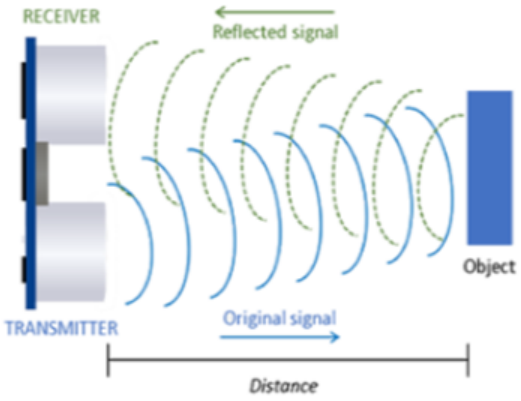


Figura 6: Eco y emisión de un frente de onda del sensor HC.SR04. [6]

Para calcular la distancia del sensor al objeto se sigue la siguiente ecuación:

$$L(cm) = \frac{tiempo(\mu s) \times 0.0343}{2} \qquad (1)$$

2.5. Puente H (L298N):

El módulo L298N posee dos canales de puente H, pudiéndolos utilizar para controlar dos motores DC o un motor paso a paso, controlando el sentido de giro y la velocidad. Está conformado por un driver L298N, sus

diodos de protección y un regulador de voltaje de 5V (78M05).

Posee un conector de 6 pines para ingresar las señales TTL para controlar los motores, una bornera de tres pines para la alimentación, y dos borneras de 2 pines para la salida a los motores.

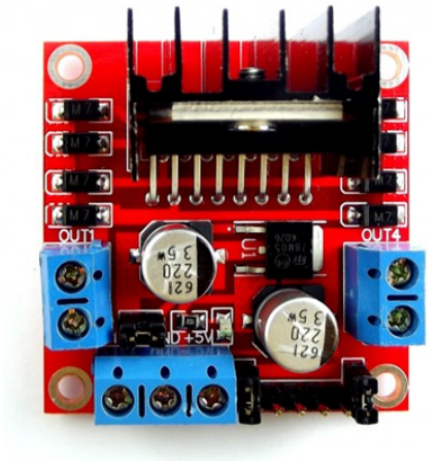


Figura 7: Puente H - módulo L298N. [7]

Empecemos explicando la forma de alimentar el módulo. Hay dos formas de hacer esto:

1. **Utilizando una sola fuente**: Conectada a la entrada de 12V y con el jumper para habilitar el regulador, aclarando que el voltaje de la fuente es el que soporta el motor. De esta forma, la entrada de 5V no debe estar conectada a ninguna fuente, ya que en este pin están presentes 5V a través del regulador interno. Sin embargo, puedes utilizar este pin como una salida de 5V, pero sin exceder los 500mA de consumo. Se recomienda hacer esta conexión para voltajes menores de 12V para no sobrecalentar el regulador.

2. **Utilizando dos fuentes**: Una de 5V conectada a la entrada de 5V (puede ser los 5V de un Arduino) y otra fuente con el valor del voltaje que requiere el motor, conectada al pin de 12V. Para esto, se tiene que desconectar el jumper, lo que deshabilitará el regulador.

Para el control del Módulo:

- Los pines ENA, IN1, IN2 corresponden a las entradas para controlar el MOTOR A (OUT1 y OUT2).
- De igual manera, ENB, IN3, IN4 permiten controlar el MOTOR B (OUT3 y OUT4).
- ENA y ENB sirven para habilitar o deshabilitar sus respectivos motores. Generalmente, se utilizan para controlar la velocidad, ingresando una señal de PWM por estos pines. Si no se usan, se deben conectar los jumpers para que siempre estén habilitados.

2.6. Motor Reductor de 6V:

El motor reductor es un dispositivo electromecánico de corriente continua con eje dual que permite reducir la velocidad y aumentar el torque. El principio de trabajo de este dispositivo se basa en sus engranajes internos (ver fig. 8).

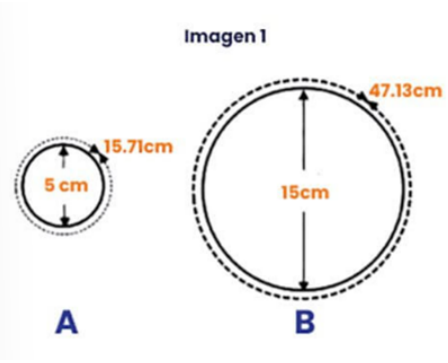


Figura 8: Esquema general de un motor reductor. [8]

Cuando gira la rueda A, hará que a su vez gire la rueda B, pero sucederá que por cada 3 vueltas que dé A, la rueda B solamente dará una vuelta. Esto es debido a la relación entre diámetros. Con esta simple combinación, se ha logrado disminuir la velocidad de rotación de la rueda B. Al agregar ruedas adicionales, podremos lograr velocidades muy pequeñas. Además, aumentaremos el torque de la última rueda.



Figura 9: Motor Reductor de 6v - 200RPM. [8]

3. Procedimiento experimental

Para la implementación de los **motores reductores** con los **puentes H**, que en conjunto servirán para hacer girar las ruedas traseras del vehículo (ver subsección 2.5), se definieron variables *IN1, IN2, IN3* e *IN4* (líneas 18 – 24 de listing 1) y se configuraron como salidas digitales (líneas 30 – 36 de listing 1) en sus respectivos pines digitales en la placa de arduino.

```
1  #include <Servo.h>
2  // Variables para el servomotor:
3  Servo MotorSensor;
4  int PinServo = 11;
5  int angulo = 0;
6  int velocidad;
7
8  // Variables para el sensor
       Ultrasonico
9  const int Trigg = 12; // Trigger
10 const int Eco = 8;
11 const float v = 0.0343;
12 const int DistSen = 25;
13 float distancia, tiempo;
14 float LeftDist, RigthDist;
15
16 // Pines para el motor reductor 1:
17 int IN1 = 2;
18 int IN2 = 7;
19 // Pines para el motor  reductor 2:
20 int IN3 = 4;
21 int IN4 = 3;
22
23 void setup() {
24   pinMode(Trigg, OUTPUT);
25   pinMode(Eco, INPUT);
26   // Configuracion de los pines de
         los motores-reductores
27   pinMode(IN1, OUTPUT);
28   pinMode(IN2, OUTPUT);
29   pinMode(ENA, OUTPUT);
30   pinMode(IN3, OUTPUT);
31   pinMode(IN4, OUTPUT);
32   pinMode(ENB, OUTPUT);
33   // Pin que usara el servomotor
34   MotorSensor.attach(PinServo);
35 }
```

Listing 1: Definición de variables y configuración de los pines para cada módulo usado.

3.1. Adquisición de datos

Las variables y funciones necesarias para la ejecucion de esta etapa se muestran en el listing 1; esta etapa se conforma por el **sensor ultrasónico HC-SR04** (líneas 1 – 6 y 33) y el **servomotor SG90** (líneas 9 – 14 y 24 – 25).

```
1  float MedirDistancia(){
2    // Generando el pulso de Trigger:
3    digitalWrite(Trigg, HIGH);
4    delayMicroseconds(10);
5    digitalWrite(Trigg, LOW);
6    // Calculando la distancia frente
         al carro:
7    tiempo = pulseIn(Eco, HIGH);
8    distancia = tiempo*v/2;
9    return distancia;
10   }
11 float GirarIzquierdaSensor(){
12   angulo=170;
13   MotorSensor.write(angulo);
14   delay(750);
15   distancia = MedirDistancia();
16   return distancia;
17   }
18 float GirarDerechaSensor(){
19   angulo=10;
20   MotorSensor.write(angulo);
21   delay(750);
22   distancia = MedirDistancia();
23   return distancia;
24   }
```

Listing 2: Funciones creadas para el registro de la distancia al frente y a los costados del vehículo.

Como se muestran en el listing 2, la función *MedirDistancia()* extrae la distancia frente al

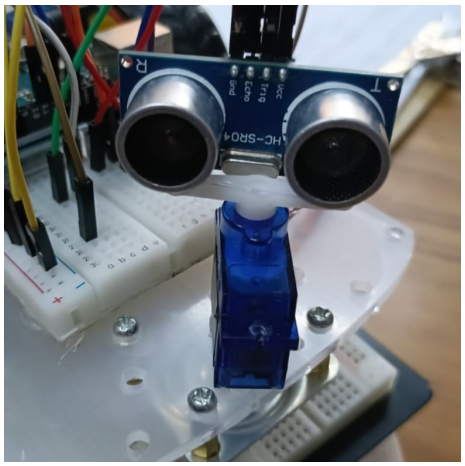


Figura 10: Montaje del servomotor con el sensor ultrasónico para el registro de distancias.

sensor.
Las funciones *GirarIzquierdaSensor()* y *GirarDerechaSensor()* hacen que el servomotor (acoplado con el sensor ultrasónico) giren hasta 170° y 10° respectivamente, apuntando a los costados y registrando las distancias en cada lado llamando a la función previamente definida *MedirDistancia()*. Cabe destacar que es necesario ubicar un delay de al menos 750ms previo a cada lectura de distancia, esto debio al tiempo que le toma al servomotor girar a los costados.

3.2. Procesamiento

En esta etapa los *datos registrados anteriormente se clasifican* según la condicional mostrada en la línea 8 del Listing 3, si la distancia que registra frente al vehículo es menor a la distancia de sensibilidad preestablecida en la línea 12 del listing 1 (mediante la variable *DistSen*).

Luego se hace un llamado a las funciones GirarIzquierdaSensor() y GirarDerechaSensor() para obtener las distancias a sus costados, estableciendo un delay de 1.5s para que los giros del servo no sean instantáneos y se obtengan lecturas correctas. Estas últimas lecturas son los *datos clasificados* que se usarán en la siguiente etapa.

```
1  void loop() {
2    distancia = DistSen+1;
3
4    // Se inicia el movimiento
         centrando el sensor
5    CentrarSensor();
6    // Comienza el movimiento
7    avanzar();
8
9    // Registrando las distancias
         frente al vehiculo
10   distancia = MedirDistancia();
11   Serial.println(distancia);
12   if( distancia <= DistSen){
13     detener();
14     delay(500);
15     LeftDist=GirarIzquierdaSensor();
16     delay(1500);
17     RigthDist=GirarDerechaSensor();
18     delay(1500);
19     CentrarSensor();
20     delay(500);
21   }
22 }
```

Listing 3: **Primera parte** del código repetitivo durante el tiempo de ejecución del vehículo.

Como se puede observar en la línea 2, la variable *ditancia* se pone como una unidad mayor a la distancia de sensibilidad *DistSen*, esto con el fin de evitar repeticiones de datos en cada ejecución de la función principal loop.

3.3. Control

Las funciones quese encargan del movimiento del vehículo son las mostradas a continuacion en el listing 4 a continuacion:2.5):

Donde los pines *IN1, IN2* pertenecen al

ENA /ENB	IN1 /IN3	IN2 /IN4	Motor Derecho	Motor Izquierdo
1	0	0	0	0
1	1	0	Horario	0
1	0	1	0	Antihorario
0	X	X	0	0

Cuadro 1: Tabla de Verdad del Módulo L298N Puente H

motor 1 (rueda derecha) y los pines *IN3, IN4,* al motor 2 (rueda izquierda). Dichas funciones se diseñaron según la siguiente tabla de verdad 1 (del puente H visto en la subsección 2.5):

```
1  void detener(){
2    digitalWrite(IN1, LOW);
3    digitalWrite(IN2, LOW);
4    digitalWrite(IN3, LOW);
5    digitalWrite(IN4, LOW);}
6  void giro_izquierda() {
7    digitalWrite(IN1, HIGH);
8    digitalWrite(IN2, LOW);
9    digitalWrite(IN3, LOW);
10   digitalWrite(IN4, HIGH);}
11 void giro_derecha() {
12   digitalWrite(IN1, LOW);
13   digitalWrite(IN2, HIGH);
14   digitalWrite(IN3, HIGH);
15   digitalWrite(IN4, LOW);}
16 void avanzar() {
17   digitalWrite(IN1, HIGH);
18   digitalWrite(IN2, LOW);
19   digitalWrite(IN3, HIGH);
20   digitalWrite(IN4, LOW);}
21 void retroceder() {
22   digitalWrite(IN1, LOW);
23   digitalWrite(IN2, HIGH);
24   digitalWrite(IN3, LOW);
25   digitalWrite(IN4, HIGH);}
```

Listing 4: Funciones creadas para el movimiento del vehículo esquivador.

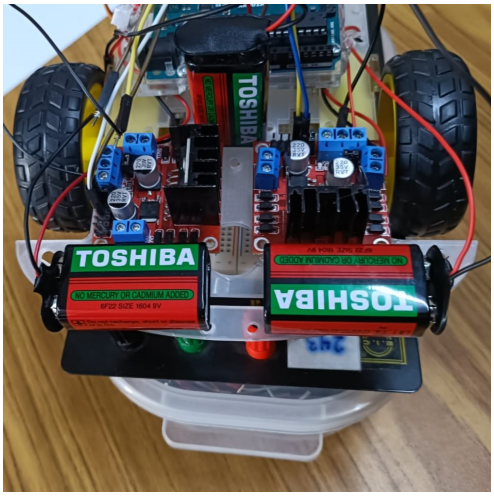


Figura 11: Montaje de 2 puentes H para cada motor reductor del vehículo esquivador de obstáculos.

Usando los datos clasificados en la etapa anterior, *se controla el movimiento del vehículo* de acuerdo a las condiciones mostradas en las lineas 3, 8 y 13 del listing 5, respectivamente si las distancias a sus costados es menor a la distancia de sensibilidad, se hace retroceder al vehículo durante $0.325s$; si la distancia a la derecha es mayor a la distancia de sensibilidad, se girará a la derecha y avanzará en la próxima ejecución del *void loop*; del mismo modo, si la distancia a la izquierda resulta se mayor a la distancia de sensibilidad, girará a la derecha y avanzará. Luego se repite indefinidamente la función *void loop* hasta que se apague el vehículo.

```
1  void loop() {
2    // Condicionando el movimiento del
         vehiculo
3    if(RigthDist <= DistSen &&
        LeftDist <= DistSen){
4      retroceder();
5      delay(325);
6      RigthDist=GirarDerechaSensor();
7      LeftDist=GirarIzquierdaSensor()
          ;}
8    if(RigthDist > DistSen &&
        LeftDist <= DistSen){
9      giro_derecha();
10     delay(325);
11     detener();
12     delay(500);}
13   if(RigthDist <= DistSen &&
        LeftDist > DistSen){
14     giro_izquierda();
15     delay(325);
16     detener();
17     delay(500);}
18   detener();
19   RigthDist = DistSen+1;
20   LeftDist = DistSen+1;
21 }
```

Listing 5: **Segunda parte** del código repetitivo durante el tiempo de ejecución del vehículo.

Como se puede observar en las líneas 19 y 20, las variables *RigthDist* y *LeftDist* se ponen como una unidad mayor a la distancia de sensibilidad *DistSen*, esto con el fin de evitar repeticiones de datos en cada ejecución siguiente de la función principal loop.

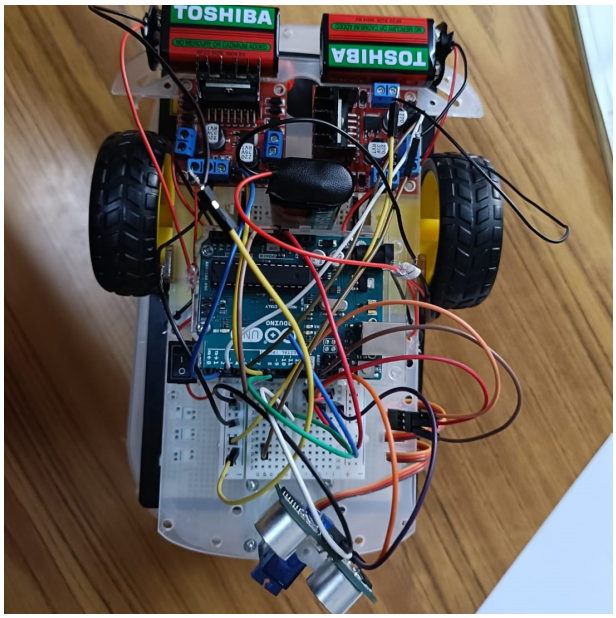


Figura 12: Montaje final del vehículo esquivador de obstáculos.

4. Aplicaciones

Domótica, automatización y control aplicados a la vivienda que se realiza mediante equipos que disponen de capacidad para comunicarse interactivamente entre sí, con capacidad de seguir instrucciones gracias a un algoritmo o programa previamente implementado y establecido por el usuario de la vivienda.



Figura 13: Aspiradora robot. [9]

5. Observaciones

- Originalmente este proyecto usaba un único puente H para controlar los motores del vehículo esquivador de obstáculos; sin embargo, debido a complicaciones con un motor reductor, se adquirió un nuevo par de motores reductores y se midió la corriente dc en cada uno, resultando 0.064A para el motor derecho y 0A para el izquierdo, es decir, que con un único puente H, solo gira una rueda del vehículo debido a la insuficiente corriente que suministra el puente H a los motores.
- Se probaron las entradas ENA y ENB de los motores reductores, con señales PWM desde 150 hasta 255, pero se observó que el motor no generaba suficiente fuerza para mover el vehículo cuando la señal PWM era menor a 180, por lo que se optó por usar el jumper habilitador (conectar a los 5V cada motor) en cada puente H.

6. Discusión y conclusiones

- Debido a las observaciones indicadas previamente en la sección 5, se usó un puente H para cada motor reductor y cada uno con su propia alimentación de 9V, obteniendo así suficiente corriente para hacer girar ambas ruedas; obteniendo así un vehículo esquivador de obstáculos funcional.
- Se logró dominar los temas estudiados durante el curso este proyecto de modo que se pueden desarrollar proyectos que beneficien a nuestra sociedad.

Referencias

[1] Arduino. Arduino uno r3 datasheet. https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf, 2024.

[2] emariete. ¿qué es una señal pwm (pulse width modulation o modulación por anchura de pulsos)? https://emariete.com/senal-pwm-pulse-width-modulation/, 2024.

[3] NaylampMechatronics. Micro servo sg90 1.5kg. https://naylampmechatronics.com/servomotores/21-servo-sg90-1kg.html, 2024.

[4] lastminuteengineers. How servo motor works and interface it with arduino. https://lastminuteengineers.com/servo-motor-arduino-tutorial/, 2024.

[5] amelectronicperu. Sensor de ultrasonido hc-sr04. https://amelectronicperu.com/producto/sensor-de-ultrasonido-hc-sr04/, 2024.

[6] QBProfeAcademy. Tutorial de arduino con sensor ultrasónico hc-sr04. https://www.qbprofe.com/automatizacion-instrumentacion-industrial/tutorial-de-arduino-con-sensor-ultrasonico-hc-sr04/, 2024.

[7] naylampmechatronics. Driver puente h l298n 2a. https://naylampmechatronics.com/drivers/11-driver-puente-h-l298n.html, 2024.

[8] naylampmechatronics. Motor dc tt 6v/200rpm eje doble de metal. https://uelectronics.com/producto/motorreductor-amarillo-para-carrito/, 2024.

[9] electrolux. Aspiradora robot electrolux). https://www.electrolux.com.pe/aspiradora-robot-erb40/p?idsku=545gad_source=1, 2024.